



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 2 - Blockchain Structure & Data Model



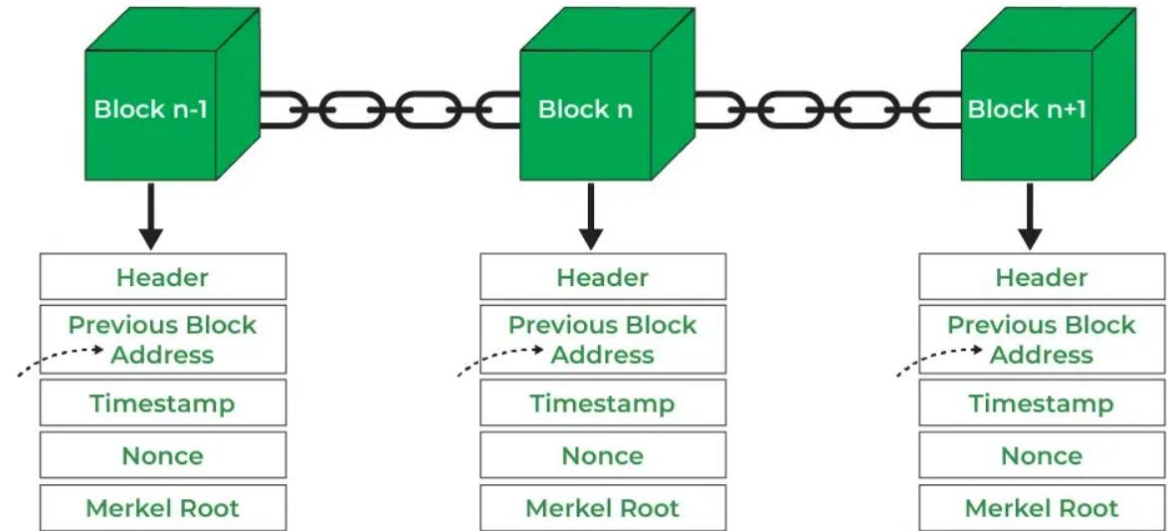
Blockchain Structure & Data Model



- 1. Blockchain as a Data Structure**
- 2. What is a Transaction?**
- 3. Hashing**
- 4. What is a Block?**
- 5. Components of a Block**
- 6. Discussion**

1. Blockchain as a Data Structure

- ❖ Blockchain is an ordered list of blocks
- ❖ New blocks are added sequentially
- ❖ Existing data is not modified (append-only)



Q: Which data structure is similar to a blockchain?

1. Blockchain as a Data Structure

Blockchain: A Probabilistic State Machine

- ❖ **Beyond a List:** While a blockchain is an ordered list of blocks, technically it is a Probabilistic State Machine
- ❖ **State Transitions:** Every transaction represents a request to change the current state (e.g., Alice's balance: $10 \rightarrow 5$, Bob's: $0 \rightarrow 5$)
- ❖ **Append-Only & Irreversible:** Data is added sequentially. Once added, a record is considered irreversible because it is linked cryptographically to every block that follows it

2. What is a Transaction?

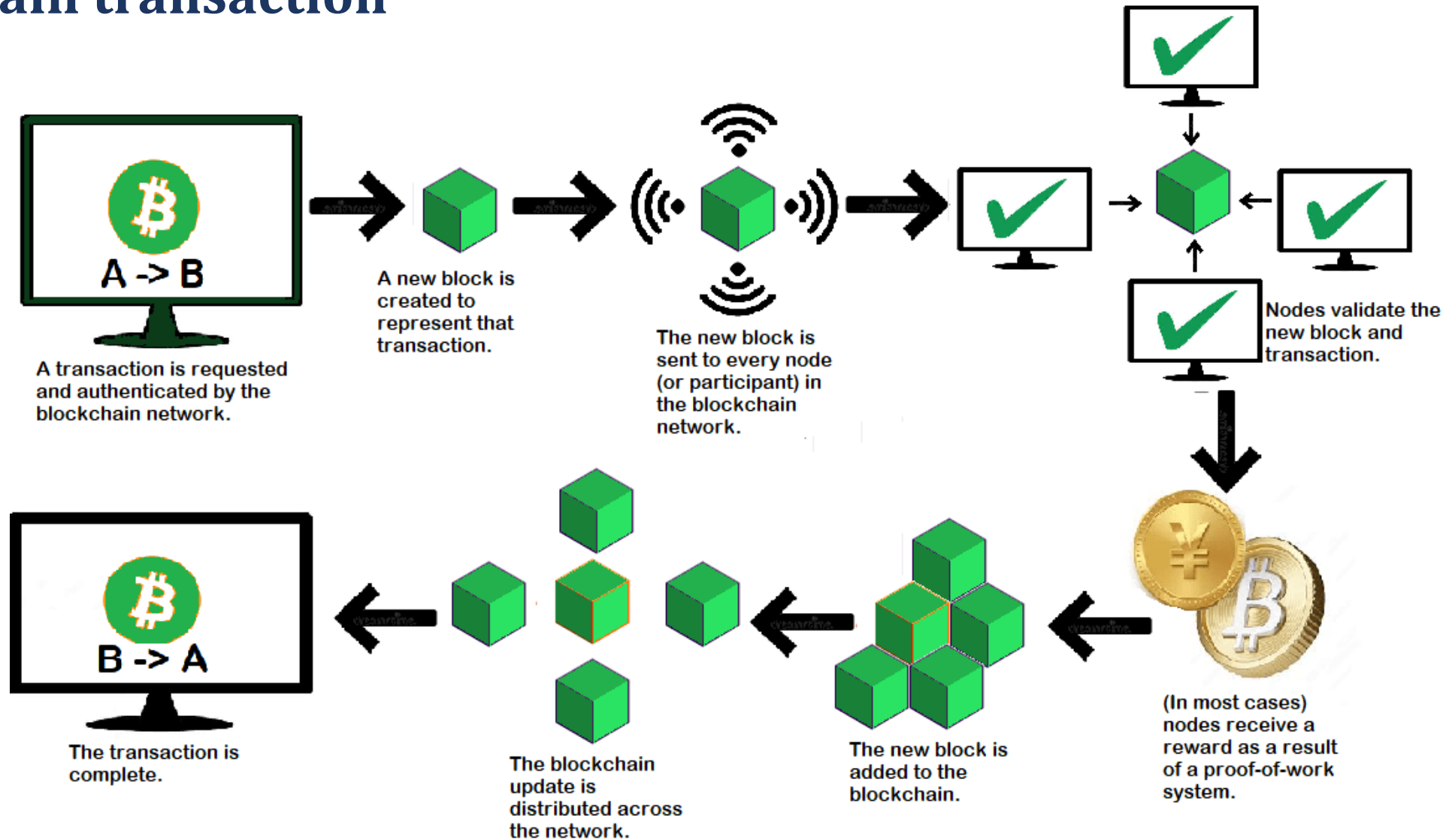


- ❖ A transaction is a request to change blockchain state
- ❖ Example: transfer value, store data
- ❖ Transactions are created by users

Q: Does every user transaction immediately become permanent?

2. What is a Transaction?

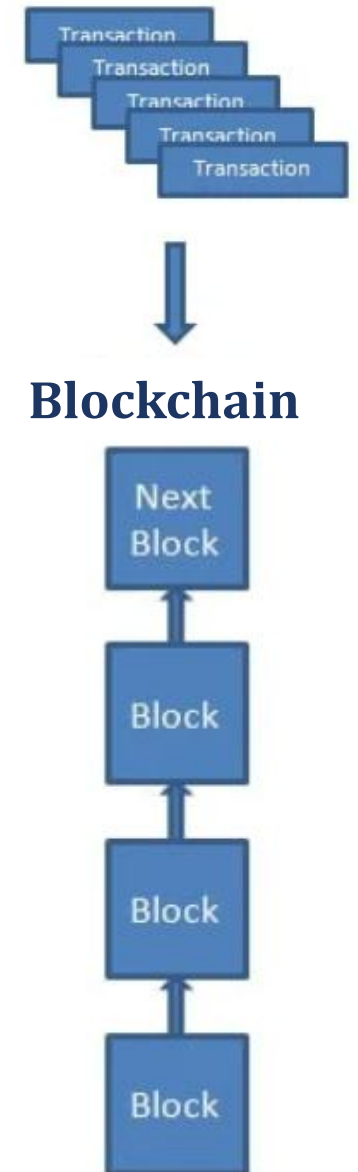
Blockchain transaction



2. What is a Transaction?

From Transactions to Blocks

- ❖ Users create and broadcast transactions to the blockchain network
- ❖ Nodes receive transactions and temporarily store them
- ❖ Transactions are checked for basic validity
- ❖ A group of valid transactions is selected and grouped into a block
- ❖ The block is then prepared to be added to the blockchain.



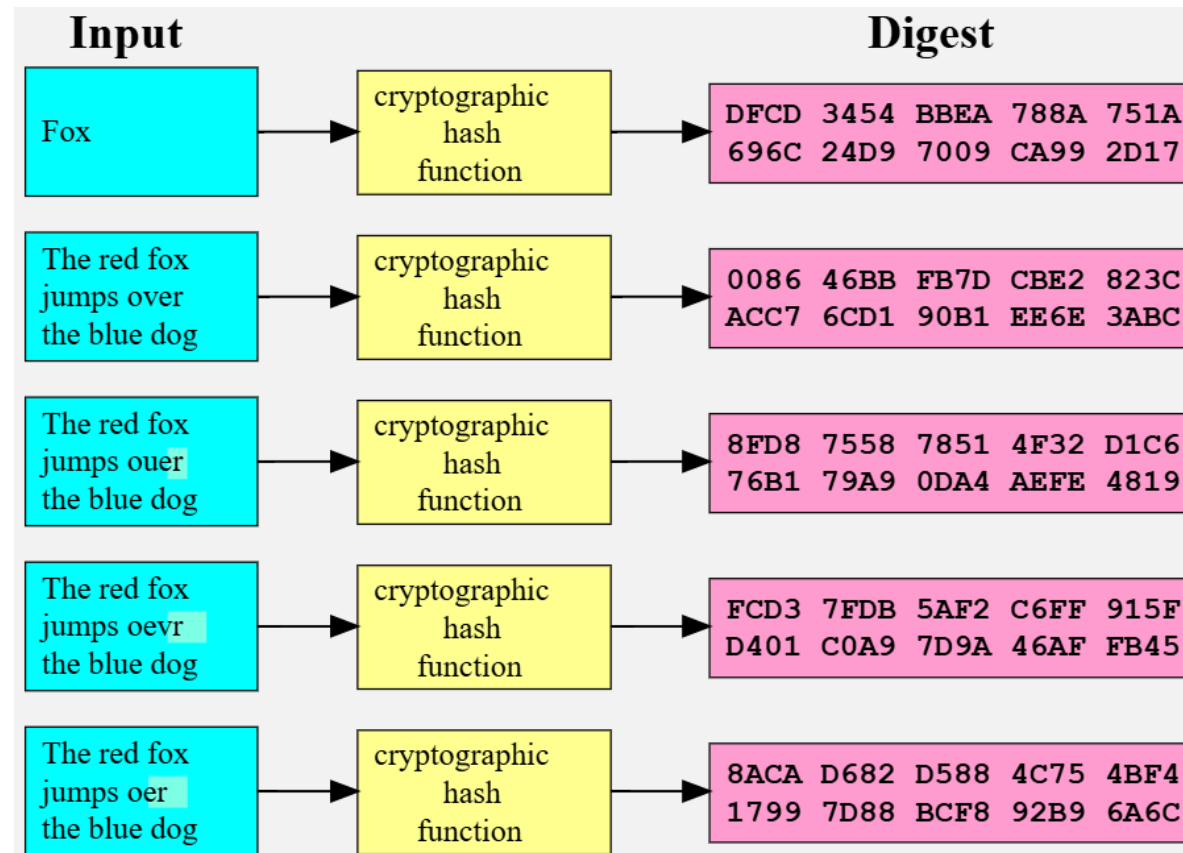
Q: Why do nodes store transactions temporarily instead of writing them directly to the blockchain?

3. Hashing



A hash function:

- ❖ Takes input data of any size
- ❖ Produces a fixed-length output (hash)



3. Hashing

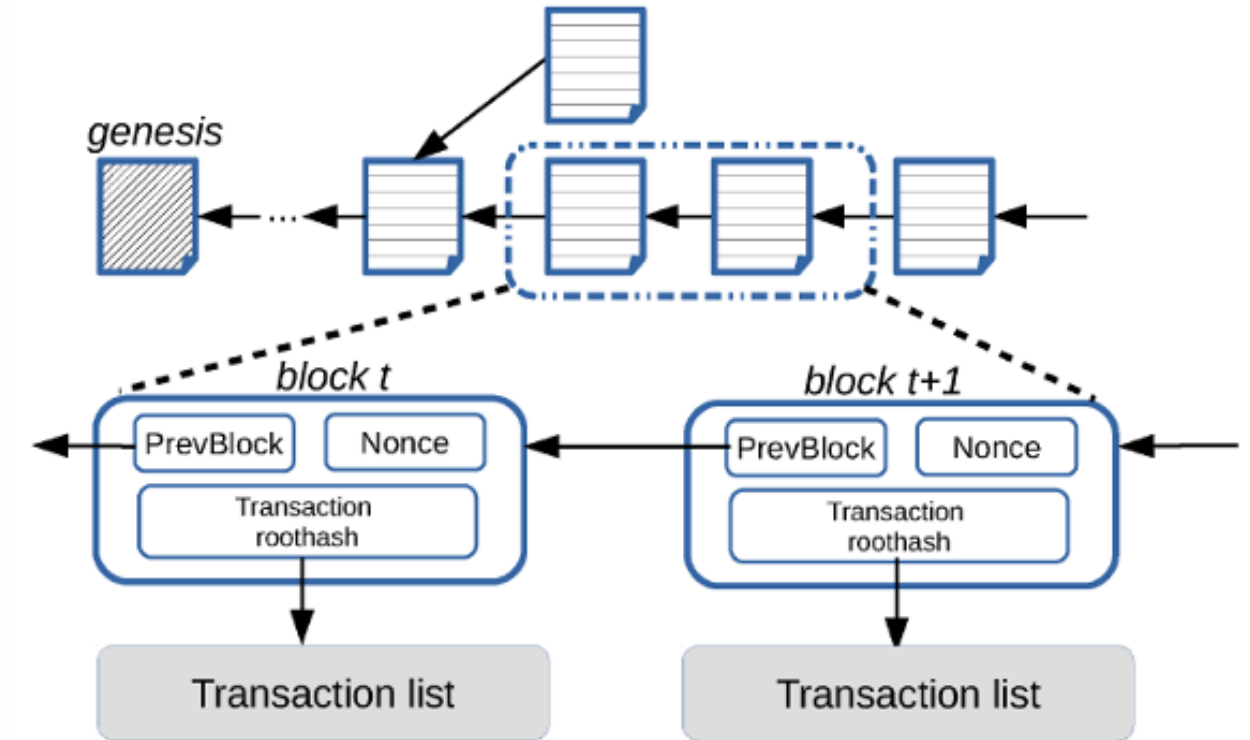


- ❖ Each transaction is hashed
- ❖ The hash represents the transaction uniquely
- ❖ Any change in transaction data changes the hash

Q: Why do we hash transactions instead of storing raw transaction data directly?

4. What is a Block?

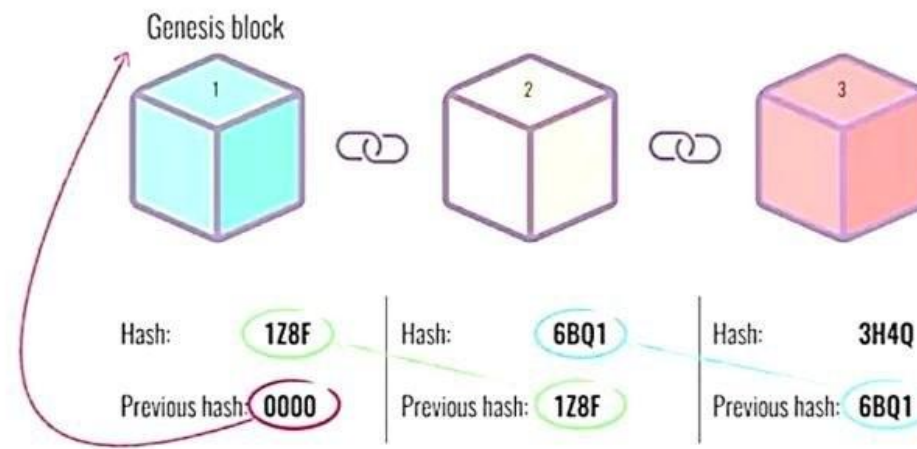
- ❖ A block is a container of transactions
- ❖ Logical unit of the blockchain
- ❖ Two main parts:
 - Block header
 - Block body
- ❖ A block is like a page in a ledger book.



4. What is a Block?

Genesis Block: The starting point of every Blockchain

- ❖ Every blockchain must begin from a **known, trusted initial state**.
- ❖ The **Genesis Block** is Block 0.
- ❖ It has **no previous block**, so its **Previous Block Hash** is conventionally set to all zeros (000...000).

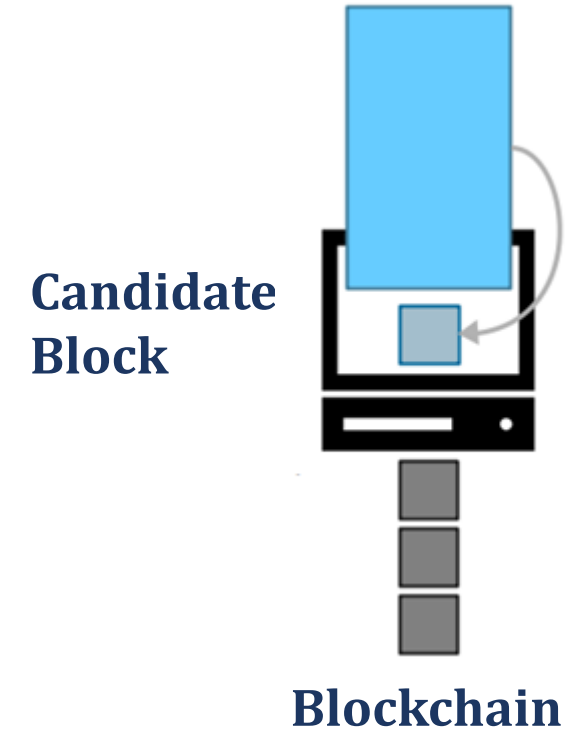


Q: Does the Genesis Block contain real transactions?

4. What is a Block?

Candidate Block

- ❖ A candidate block is a temporary block created from a set of valid transactions
- ❖ Transactions are selected from a node's temporary transaction storage
- ❖ Each participating node may construct its own version of a candidate block
- ❖ The candidate block is not yet part of the blockchain
- ❖ Only after validation and agreement can a candidate block become a confirmed block



Q: Can two nodes create different candidate blocks at the same time?

4. What is a Block?

Adding a new block to the Blockchain:

- ❖ Each node then tries to add their candidate block to the blockchain through the process of mining.
- ❖ Every block starts life as a candidate block, but only the ones that are successfully mined get added to the blockchain.
- ❖ Each miner starts working on a new candidate block, as soon as they hear about the latest one being found

5. Components of a Block

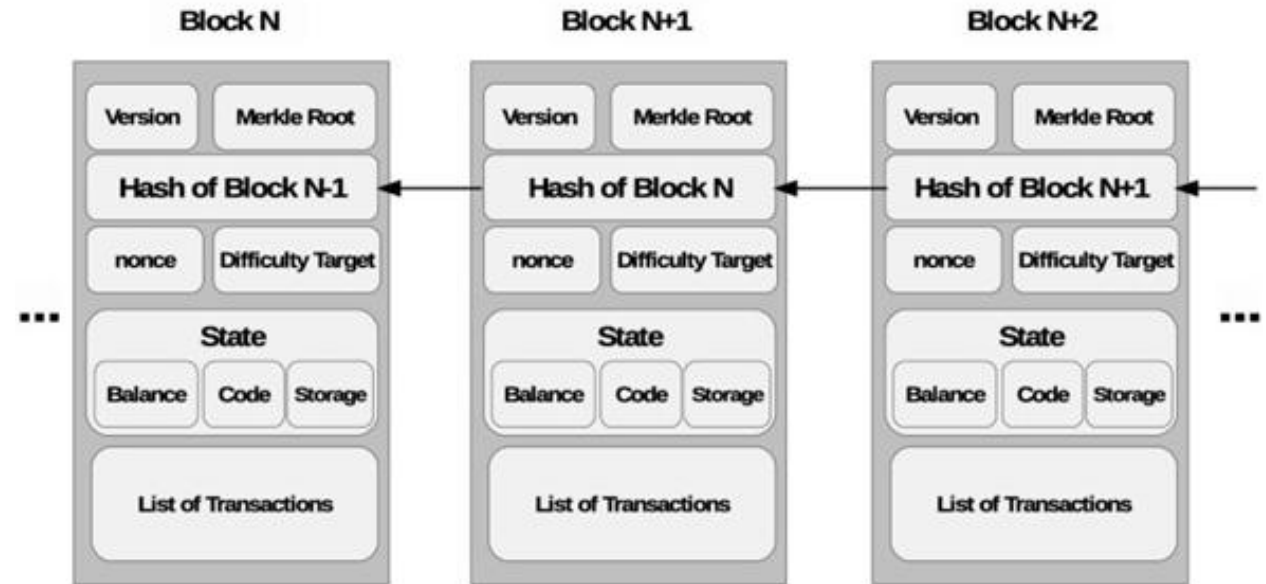
Block Structure: Header and Body

❖ Block Header:

- Previous Block Hash
- Merkle Root of Transactions
- Timestamp
- Other Metadata: Nonce, difficulty target (depending on blockchain)

❖ Block Body

- List of Transactions



5. Components of a Block

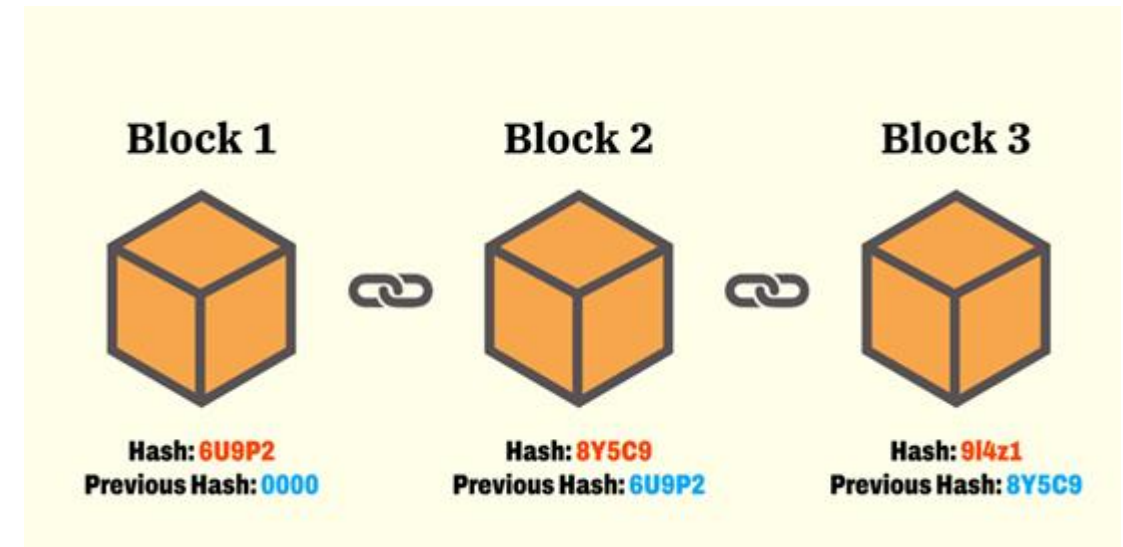
Block Body: Transactions

- ❖ Contains a list of validated transactions
- ❖ Transactions represent state changes
- ❖ Size of block body may vary

5. Components of a Block

Previous Block Hash

- ❖ Each block header contains the hash to the previous block
 - Ensures order and integrity of blocks: This is the **chain** in blockchain
- ❖ Hash acts as a digital fingerprint
- ❖ Without this component, there would be no connection and chronology between each block



Q: Why does the header include the previous block's hash?

5. Components of a Block

Block Header: Metadata

- ❖ Version: Rules being followed by the node
- ❖ Merkle Root: A single 32-byte hash representing every transaction in the block body
- ❖ Timestamp: The approximate time the block was created
- ❖ Nonce (4 bytes): A random number miners change to find a valid hash
- ❖ Difficulty Target: The threshold value the block hash must be smaller than to be valid

Example: *If you change a single comma in transaction #50, the Merkle Root changes, which changes the Block Header, which invalidates the block hash and every block after it.*

5. Components of a Block

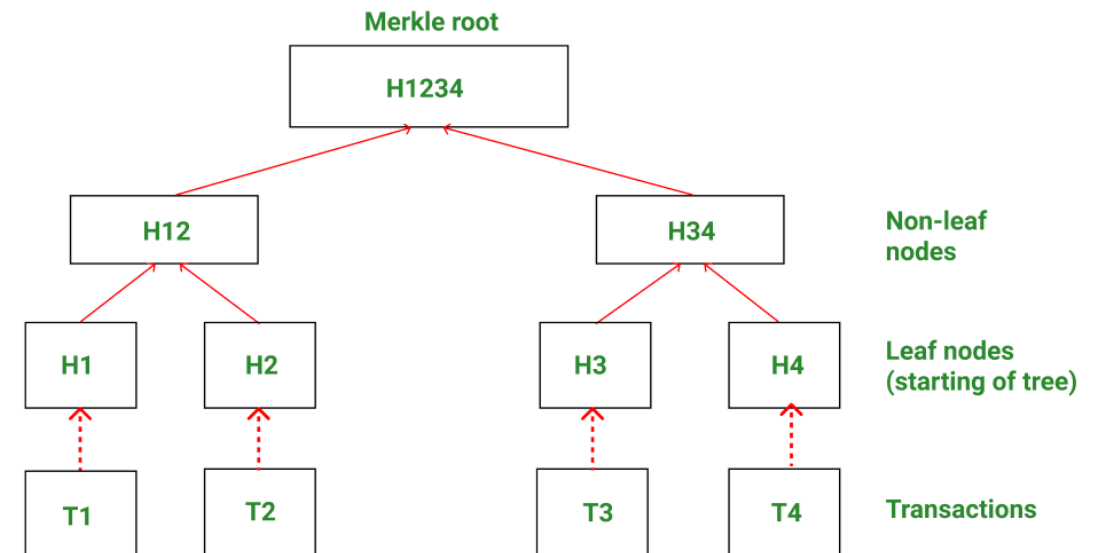
Merkle Root

- ❖ Transactions in a block are organized using a Merkle tree
- ❖ A Merkle tree is a hash-based tree structure
- ❖ The Merkle root is a single hash that represents all transactions in the block
- ❖ The Merkle root is stored in the block header
- ❖ Any change to a transaction results in a different Merkle root

5. Components of a Block

Why Merkle Tree is Important

- ❖ Merkle Tree allows:
 - Efficient transaction verification
 - Small data proofs
 - Fast detection of data changes



Q: Why not just hash all transactions together once?

5. Components of a Block

Merkle Trees: Recursive Hashing & Proof of Membership

- ❖ Transactions are hashed into unique IDs (TxIDs).
- ❖ TxIDs are paired, concatenated, and hashed again using SHA-256 until only one hash remains: the **Merkle Root** (final single hash located at the very top of this tree structure)
- ❖ Edge Case: If a block has an odd number of transactions, the last hash is duplicated to form a pair
- ❖ Mathematical Efficiency
 - The verification cost is logarithmic: **$O(\log_2(n))$**
 - To verify 1 transaction among 1,024, a node only needs 10 specific hashes to prove it matches the Merkle Root stored in the header.

6. Discussion

1. Using your own words, explain how a Merkle tree works in a blockchain. Draw a simple Merkle tree diagram to present your explanation.
2. How does the Merkle tree help maintain blockchain security and integrity?
3. Why not just hash all 2,000 transactions together in one big string instead of using a Merkle Tree?